

Hadoop & MapReduce: Distributed Processing for Unstructured Data

BDA POP — Session 09/15/2026

Why this document exists

A platform the size of Zomato or Swiggy doesn't just store structured rows like cost and rating — it accumulates unstructured text at the same rate: millions of customer reviews and order notes a day, in free-form language, with no fixed schema. A single machine running a script has two hard limits: how much data fits in its RAM, and how fast one CPU can walk through it. Text at real platform volume routinely blows past the first limit long before the second one matters. Hadoop exists to remove that ceiling — spread the data itself across many machines' disks, then run the computation on each machine where its own slice of data already lives, instead of hauling everything to one place first. Production systems at this scale aren't hypothetical: Google's MapReduce jobs have routinely processed multi-terabyte web indexes across tens of thousands of servers, and Facebook, Yahoo, and Twitter all built data-intensive pipelines on Hadoop. The canonical MapReduce example — counting words across a huge document set — is the same shape of problem as tallying sentiment words across a huge pile of reviews, which is what this document builds.

What MapReduce actually does

MapReduce breaks a job into three phases. **Map** splits the input into pieces and runs the same function on each piece independently, producing key/value pairs — here, a word and a count of 1 for each time it appears. **Shuffle & sort** groups every pair by key across all the pieces, so all the counts for the same word end up together regardless of which piece they came from. **Reduce** combines the grouped values into a final answer per key — here, the total count per word. The map step never needs to see the whole dataset, only its own piece, which is exactly what lets it run on many machines at once.

That independence pays off a second way: because map and reduce functions are deterministic — the same input block always produces the same output — a task that fails or a machine that dies mid-job can simply be re-run somewhere else. Nobody has to figure out what partial state to salvage; the framework just redoes that one piece. Ten mappers running, two crash: the framework reschedules those two, the other eight keep going, and the final answer is unaffected. That's the second reason this model scales — not just that work splits cleanly, but that failure is cheap to recover from.

Hadoop's architecture underneath MapReduce

MapReduce is the computation model; Hadoop is the system that runs it at cluster scale, built from two other pieces.

HDFS (Hadoop Distributed File System) splits a file into fixed-size blocks — 128 MB by default, deliberately large — and spreads them across the disks of many machines (DataNodes). A NameNode holds only the metadata: which blocks make up which file, and which DataNodes hold each one; it never stores file data itself. The 128 MB default isn't arbitrary: fewer, bigger blocks means far less metadata for the NameNode to track per file, and HDFS is optimized for streaming large files end-to-end, not for seeking to small random offsets (that's HBase's job, below). Each block is replicated three times by default, placed on different DataNodes and — where the cluster spans multiple racks — on different racks, so losing an entire rack, not just a single disk, still leaves two copies standing. DataNodes heartbeat to the NameNode continuously; if one goes silent, the NameNode marks its blocks under-replicated and schedules fresh copies onto healthy nodes, automatically, with no human paged. The honest caveat: the NameNode itself is a single point of failure in this design — production clusters run NameNode high-availability pairs specifically because of it.

YARN (Yet Another Resource Negotiator) is what decides which machine runs which task and how much CPU/memory it gets, split across three roles: one **ResourceManager** per cluster tracks all available capacity and hands out containers; one **NodeManager** per machine runs and monitors the containers assigned to it and reports back; and one **ApplicationMaster** per submitted job negotiates that job's own containers from

the ResourceManager and supervises only that job's tasks, restarting the ones that fail. Hadoop 1.0 used a single JobTracker doing all of this at once and hit scaling limits; Hadoop 2.0 split it into these three roles specifically to fix that. The payoff: because YARN is a general resource negotiator and not something wired to MapReduce specifically, Spark, Hive, and other engines can all run on the same cluster, sharing the same capacity, instead of each needing its own dedicated hardware.

The payoff of putting HDFS and YARN together: when a MapReduce job runs, the ResourceManager schedules each map task's container on — or near — the DataNode that already holds that task's input block. The computation moves to the data, not the other way around. That's the part a single-machine script structurally cannot do, no matter how it's written.

The ecosystem built on top: Hive, Pig, and HBase

Raw MapReduce is what makes distributed computation possible, but almost nobody at Zomato-or-Swiggy scale writes raw Map and Reduce functions for everyday work. Three tools sit on top of HDFS+YARN and cover the jobs a hand-written MapReduce job would be painfully slow to build:

Tool	What It Gives You	Runs As
Hive	SQL-like queries (HiveQL) over data already sitting in HDFS	Compiled down to MapReduce/Tez jobs
Pig	A scripting language (Pig Latin) for multi-step ETL pipelines: load, filter, group, join	Compiled down to MapReduce jobs
HBase	Real-time, random read/write access to individual rows (NoSQL)	Its own service layer on HDFS — no MapReduce per lookup

Hive gives analysts a query language over data that's already sitting in HDFS, compiled behind the scenes into a MapReduce job the analyst never has to see. Without it, "how many one-star reviews came from restaurants in Bangalore last month?" is a ticket to an engineer, not a query someone runs themselves before lunch. **Pig** gives data engineers a pipeline language for the daily job that turns yesterday's raw, messy review and order logs into the cleaned table Hive queries run against — without it, every format change in the raw logs means hand-editing bespoke MapReduce code instead of one pipeline script.

HBase is the exception to all of it. HDFS+MapReduce are built to scan huge files in bulk; a MapReduce job to find "this one customer's most recent order" would mean scanning blocks across the whole cluster for a single row. HBase is a NoSQL, column-oriented store modeled on Google's Bigtable that answers exactly that kind of point lookup in milliseconds — the difference between a support agent pulling up an order live on a call versus reading from a report that's hours old. Hive and Pig still compile down to the same batch MapReduce model described above; HBase steps outside it entirely, because "wait for a batch job" isn't good enough for a live lookup.

Why not just buy a bigger machine?

The instinctive alternative to all of this is vertical scaling — more RAM and a faster CPU on one box. It has a hard ceiling: even the largest commodity servers top out well under real platform text volume, and cost climbs faster than capacity as that ceiling approaches. Horizontal scaling — more ordinary machines instead of one bigger one — has no fixed ceiling: need more capacity, add more nodes. That choice is what Hadoop's entire architecture is built around.

The dataset

This week extends the same 200-restaurant Zomato/Swiggy dataset used in every prior session (`set.seed(42)`, zero external files) with a new `review_text` column — 40 free-text words per restaurant, drawn from a small vocabulary weighted so higher-rated restaurants generate more positive words. That gives the corpus 8,000 words total: small on purpose, to keep the demo instant. The lesson comes from extrapolating this toy corpus to real platform volume, not from the corpus's own size.

The naive approach: everything in one machine's memory

The most direct way to count words is a single loop over every review, keeping one running tally. At 200 restaurants this finishes in 0.014 seconds — the flaw here isn't speed. It's that this function requires every review to already be sitting in one process's memory before it can start. That assumption breaks long before CPU speed becomes the bottleneck.

Projected to real platform volume — a stated assumption of 500 million reviews/day, roughly 150 bytes each, over a 90-day trailing analytics window — the corpus this approach would need to hold comes to roughly **6.75 TB**. A generous single-machine RAM ceiling of 256 GB is **26.4 times** smaller than that. No amount of loop optimization fixes that; the data has to live somewhere else from the start.

Metric	Value	Threshold	Status
90-day review volume vs. single-machine RAM	~26.4x over	≤ 1x (must fit in RAM)	Flagged

The fix: partition like HDFS would, map locally, reduce the small result

HDFS would already have this data pre-split into blocks spread across many DataNodes. The fix simulates that with R's `split()`, runs the map step on each partition independently with `mclapply()` — a stand-in for one mapper task per node — then reduces the small partial tallies, never the raw text, into one final answer. Running this on the same 200-restaurant corpus produces a word count that's `identical()` to the naive single-machine result — confirmation that the distributed rewrite doesn't change the answer, only where the computation happens.

Splitting the same 6.75 TB of projected volume across 200 nodes — a modest real Hadoop cluster — brings each machine's share down to roughly **33.75 GB**: comfortably inside the ceiling the single-machine version couldn't clear at all.

Metric	Value	Threshold	Status
Per-node shard size (200-node cluster)	~33.75 GB	≤ 256 GB RAM ceiling	Within threshold
Word-count result vs. naive single-machine run	identical	must match	Within threshold

An honest complication: distribution isn't free

The two elapsed times actually measured on the 200-restaurant, 8,000-word toy corpus tell an inconvenient truth: the single-loop naive version finished in 0.014 seconds, while the 8-way partitioned Map+Reduce version took 0.14 seconds — noticeably *longer*, not shorter. That's a real measurement, not an error. Forking worker processes and reassembling their results costs time, and at this small a scale that coordination overhead outweighs the work being distributed. The lesson isn't that MapReduce is slower — it's that distribution only pays for itself once each machine's local share of the work is large enough to be worth

coordinating. That's exactly the 26.4x-over-RAM scenario above, and exactly why nobody runs Hadoop on 8,000 words: the value shows up at the volume where a single machine can't do the job at any speed, not as a speed trick at small scale.

Where MapReduce falls short — and why the pattern outlived the engine

MapReduce's honest weak spot is iterative work. Every MapReduce job reads its input from disk and writes its output back to disk before the next stage can start; a machine-learning algorithm that needs a hundred passes over the same data pays that disk round-trip a hundred times, with no way to keep intermediate results in memory across passes. That's a batch model built for throughput on one large pass, not for interactive queries or repeated iteration — and it's the specific gap Spark was built to close by keeping data in memory across steps, which is why Spark displaced MapReduce for machine-learning workloads rather than replacing it everywhere.

What didn't fade is the shape of the problem. A modern batch embedding pipeline — chunk millions of documents, compute an embedding vector for each, route vectors to partitions, assemble the partitions into index shards — is map, shuffle, and reduce under a different name, tuned by the same instincts: chunk size versus coordination overhead, a shuffle step that dominates cost, tasks that re-run safely on failure because the work is deterministic. Learning MapReduce's pattern pays off even where the original engine doesn't run anymore.

Summary

Check	Result
Naive single-machine approach vs. 90-day review volume	Flagged — ~26.4x over a single machine's RAM
Partitioned Map+Reduce shard size vs. RAM ceiling (200 nodes)	Within threshold — ~33.75 GB per node
Partitioned result vs. naive result	Within threshold — identical word counts
Partitioned runtime vs. naive runtime, at toy (8,000-word) scale	Flagged — coordination overhead exceeds the naive runtime here

MapReduce's win isn't raw speed on any given machine — it's turning a job that's structurally impossible on one machine into a set of independent, combinable pieces. Hadoop's HDFS+YARN pair is what makes that split automatic: data pre-sharded across disks, computation scheduled to run where each shard already sits. Before reaching for a distributed system on a new job, the RAM-ceiling question above is the one worth asking first: does this actually not fit on one machine?